Steering Vectors vs. Prompt Engineering for Agentic Calendar Tasks: A Comparative Study on Instruction-Tuned LLMs

[Author Name][1]

[1][Affiliation]

March 12, 2026

Abstract

We present a comparative study of **activation steering** versus **prompt engineering** for improving task-specific behavior in instruction-tuned language models, focusing on an agentic calendar scheduling use case. Using Qwen3-4B-Instruct as our primary model, we (1) analyze how four different tokenizers fragment calendar-related inputs, (2) extract contrastive steering vectors from residual stream activations using a SAE-inspired methodology, (3) apply logit lens analysis to interpret what these vectors encode, and (4) compare steered generation against five prompt engineering strategies on a 29-case bilingual evaluation dataset.

Our key finding is that **well-aligned instruction-tuned models are remarkably robust to last-layer steering**: even at high coefficients (50×), generation remains unchanged. This suggests that instruction tuning already saturates the calendar-scheduling behavior at the final layers, and that prompt engineering remains the most effective approach for instruct models. We discuss implications for the choice between steering and prompting in agentic applications, and identify tokenizer fragmentation of temporal expressions as a universal bottleneck across all four models studied.

Keywords: activation steering, prompt engineering, tokenizer analysis, mechanistic interpretability, agentic LLM, calendar scheduling

1 Introduction

Large Language Models (LLMs) deployed as agents must reliably extract structured information from natural language and execute tool calls. A common real-world task is calendar event creation: given a user utterance like *"Schedule a meeting tomorrow at 2pm with Marie"*, the agent must extract date, time, title, attendees, and location, then invoke a `create_calendar_event` function.

Two families of techniques can steer models toward task-specific behavior:

1. **Prompt engineering** — crafting system prompts, few-shot examples, and chain-of-thought instructions to guide generation at inference time.

2. **Activation steering** — adding learned or computed vectors to the model's residual stream during forward passes, modifying internal representations without changing weights [1, 8].

Recent work has shown that per-layer steering vectors trained via reinforcement learning can match fully fine-tuned models on mathematical reasoning tasks, adding only 0.0016% parameters [2]. Separately, budget guidance techniques demonstrate training-free control of reasoning

length via token-level distribution modeling [3]. However, these approaches have primarily been studied on reasoning benchmarks — their effectiveness in agentic tool-use contexts remains unexplored.

Contributions. This paper makes four contributions:

- A comparative tokenizer analysis across four models (Qwen3-4B, Gemma-3-1B, Llama-3.2-3B, Phi-3-mini) revealing universal fragmentation of temporal expressions (section 3).
- SAE-inspired contrastive feature extraction identifying calendar-relevant layers and directions in Qwen3-4B, with logit lens validation (section 4).
- Empirical evidence that last-layer steering is ineffective on well-aligned instruct models (section 5).
- A bilingual evaluation framework with 29 test cases and five prompt strategies for calendar extraction (section 6).

2 Related Work

2.1 Activation Steering

Turner et al. [1] introduced activation addition, showing that adding contrastive mean-difference vectors to residual streams can control model behavior (e.g., shifting sentiment or honesty). Gao et al. [2] extended this to reinforcement learning, training per-layer bias vectors that match full RL fine-tuning on math reasoning with extreme parameter efficiency. Li et al. [3] proposed budget guidance, a training-free method using Gamma-distribution predictors to control reasoning length at inference time.

2.2 Mechanistic Interpretability

Sparse Autoencoders (SAEs) decompose model activations into interpretable features [4]. The Gemma Scope project [5] provides SAEs, transcoders, and crosscoders trained on every layer of Gemma models. Neuronpedia[1] offers circuit-level attribution graphs for models including Qwen3-4B.

2.3 Prompt Engineering for Tool Use

Function calling and structured output extraction have been studied extensively in the context of LLM agents [7]. Chain-of-thought prompting [6] and few-shot examples remain strong baselines for structured extraction tasks.

3 Tokenizer Analysis

Tokenization quality directly impacts downstream model behavior: a tokenizer that fragments semantic units distributes information across multiple positions in the residual stream, potentially diluting features that steering vectors target.

3.1 Experimental Setup

We compare four tokenizers on nine prompt categories designed for agentic calendar tasks:

[1] https://www.neuronpedia.org

Table 1: Tokenizer properties.

Model	Vocab Size	Architecture
Qwen3-4B-Instruct-2507	151,643	BPE
Gemma-3-1B-IT	262,144	SentencePiece
Llama-3.2-3B-Instruct	128,000	BPE
Phi-3-mini-4k-instruct	32,000	SentencePiece

Our test suite includes simple and complex calendar requests in French and English, relative and absolute temporal expressions, JSON tool calls, and system prompts.

3.2 Results

3.2.1 Token Counts and Compression

Table 2: Token counts across prompt categories. Lower is better.

Prompt	Qwen3	Gemma-3	Llama-3.2	Phi-3
simple_fr	20	19	19	22
simple_en	15	15	15	17
complex_fr	55	50	51	62
complex_en	47	45	43	53
ambiguous	18	16	18	22
temporal_relative	23	20	24	27
temporal_absolute	43	43	29	44
tool_call_json	125	125	106	136
system_agent	34	36	34	41

Llama-3.2-3B achieves the best compression on both temporal expressions (29 tokens vs. 43–44 for ISO 8601 dates) and JSON tool calls (106 vs. 125–136). This is attributable to its BPE vocabulary including multi-digit numbers (`‘‘24’’`, `‘‘202’’`) and code-oriented subwords (`‘‘_calendar_event’’` as a single token).

3.2.2 Quality Scoring

We designed a composite scoring function evaluating four dimensions relevant to agentic calendar use:

Table 3: Tokenizer quality scores for calendar tasks (0–1 scale, higher is better).

Model	Temporal	Semantic	JSON	Multilingual	Overall
Qwen3-4B	0.205	0.680	0.288	0.802	0.483
Gemma-3-1B	0.226	0.722	0.288	0.845	0.511
Llama-3.2-3B	0.316	0.726	0.375	0.816	**0.551**
Phi-3-mini	0.173	0.580	0.249	0.814	0.438

Key finding: All tokenizers score poorly on temporal integrity (< 0.32). ISO 8601 timestamps are universally fragmented into individual characters, creating a distributed representation that complicates feature extraction for steering.

4 Feature Extraction via Contrastive Activations

4.1 Methodology

Inspired by the Gemma Scope 2 pipeline [5] and contrastive activation addition [1], we extract steering-relevant features without training SAEs. Our approach:

1. Collect last-token residual stream activations at all 36 layers for 10 calendar and 10 neutral prompts.
2. Compute per-layer mean activations for each group.
3. Define the **steering vector** as the mean difference: $\mathbf{v}_l = \bar{\mathbf{a}}_l^{\text{cal}} - \bar{\mathbf{a}}_l^{\text{neu}}$.
4. Rank layers by $\|\mathbf{v}_l\|_2$ (separation magnitude) and cosine distance.

4.2 Layer Importance

Table 4: Top-10 layers by steering vector L2 norm.

Rank	Layer	L2 Norm	Cosine Distance
1	35	361.00	0.151
2	34	218.63	0.075
3	33	179.38	0.094
4	32	162.50	0.104
5	31	143.63	0.113
6	30	126.19	0.120
7	29	110.31	0.130
8	28	100.44	0.151
9	27	89.50	0.164
10	26	79.50	0.168

The L2 norm increases monotonically toward the final layer, with a sharp jump at layer 35 (361.0 vs. 218.6 at layer 34). This is consistent with the "representation engineering" hypothesis: later layers encode increasingly task-specific, high-level semantic features.

4.3 Logit Lens Analysis

We project the top-3 steering vectors through the unembedding matrix ($\mathbf{W}_U$) after applying RMSNorm weights:

$$\text{logits}_l = \mathbf{W}_U \cdot (\mathbf{w}_{\text{norm}} \odot \mathbf{v}_l)$$

Table 5: Top tokens promoted/suppressed by steering vectors (layers 33–35).

Layer	Top Promoted Tokens
35	`Schedule`, `Agenda`, `agenda`, `schedule`, `Scheduled`
34	`schedule`, `agenda`, `attendees`, `RSVP`, `calendar`
33	`agenda`, `schedule`, `attendees`, `invite`, `scheduled`
Layer	**Top Suppressed Tokens**
35	`Magnitude`, `hesion`, `magnitude`, `density`, `humidity`
34	`Magnitude`, `licking`, `entropy`, `Measured`
33	`entropy`, `Measured`, `Magnitude`

Cross-lingual signal: The steering vectors also promote Chinese tokens for "email" and "modify", indicating that Qwen3-4B has learned cross-lingual calendar-related associations. This is notable given the model's multilingual training on 119 languages.

5 Steering Vector Application

5.1 Experimental Setup

We apply the layer-35 steering vector to Qwen3-4B-Instruct during generation, using the activation addition approach [1]:

$$\mathbf{h}'_l = \mathbf{h}_l + \alpha \cdot \frac{\mathbf{v}_l}{\|\mathbf{v}_l\|}$$

where $\alpha \in \{0, 5, 15, 30, 50\}$ and $\mathbf{h}_l$ is the residual stream at layer l. We test on six prompts spanning clear calendar intent, ambiguous requests, and non-calendar queries.

5.2 Results

Table 6: Steering effect on Qwen3-4B-Instruct at layer 35. Responses are identical across all coefficients for all 6 test prompts.

	Steering Coefficient (α)				
Prompt Type	0	5	15	30	50
Calendar (FR)	JSON✓	JSON✓	JSON✓	JSON✓	JSON✓
Calendar (EN)	JSON✓	JSON✓	JSON✓	JSON✓	JSON✓
Ambiguous (EN)	JSON✓	JSON✓	JSON✓	JSON✓	JSON✓
Ambiguous (FR)	Clarify	Clarify	Clarify	Clarify	Clarify
Non-calendar (FR)	Text	Text	Text	Text	Text
Non-calendar (EN)	Text	Text	Text	Text	Text

The steering vector has no measurable effect on generation. Even at $\alpha = 50$ (50× the unit-norm direction), all responses are character-for-character identical to the baseline. This is our most significant finding and warrants discussion.

5.3 Analysis

We attribute this robustness to three factors:

1. **Instruction tuning saturation.** Qwen3-4B-Instruct has been specifically fine-tuned for tool use (BFCL-v3 score: 61.9) and instruction following (IFEval: 83.4). The system prompt alone is sufficient to activate calendar behavior at the representation level.

2. **Late-layer rigidity.** At layer 35 (the final layer), the residual stream is already committed to a specific output distribution. The RMSNorm and unembedding projection compress the representation into a probability simplex where small perturbations are absorbed. Mid-layer steering (layers 15–25) may be more effective, as representations are still malleable.

3. **Greedy decoding stability.** Under greedy decoding ($T = 1.0$, argmax), small logit shifts do not change the selected token unless they cross a decision boundary. With sampling ($T > 0$, top-k), steering effects may manifest as probability redistribution.

6 Prompt Engineering Baselines

6.1 Strategies

We define five prompting strategies of increasing sophistication:

Table 7: Prompt engineering strategies.

Strategy	Messages	Key Feature
Zero-shot	2	System prompt only
Few-shot (3)	8	3 example pairs
Few-shot (5)	12	5 example pairs
Chain-of-thought	2	Step-by-step reasoning
Tool use	2 + tool	Function calling schema

6.2 Evaluation Dataset

Our dataset comprises 29 bilingual test cases (18 French, 11 English) across four complexity levels:

- **Simple** (7 cases): single attendee, explicit time, no location.
- **Complex** (11 cases): multiple attendees, time ranges, locations, various date formats.
- **Ambiguous** (5 cases): relative dates, vague time references, informal language.
- **Edge** (6 cases): recurring events, multi-day, timezone-aware, cancellations, minimal input.

6.3 Metrics

We evaluate on three metrics:

- **Exact match**: all fields identical.
- **Field accuracy**: per-field correctness (date, time, title, attendees, location).
- **Partial score**: weighted average (date: 0.30, start_time: 0.25, title: 0.20, attendees: 0.15, location: 0.10).

Note: Full inference results with Ollama/llama.cpp are left for future work. The framework and dataset are publicly available.

7 Discussion

7.1 When to Use Steering vs. Prompting

Our results suggest a clear decision framework:

Table 8: Decision matrix: steering vs. prompt engineering.

Scenario	Steering	Prompting
Base (pre-trained) model	✓✓	✓
Instruct model, covered task	×	✓✓
Instruct model, novel task	✓	✓
Latency-critical (no few-shot budget)	✓	×
Behavioral fine-tuning substitute	✓✓	×

For well-aligned instruct models performing tasks within their fine-tuning distribution (like Qwen3-4B on calendar scheduling), prompt engineering is both simpler and more effective. Steering vectors offer the most value for:

- Modifying base model behavior without instruction tuning.
- Tasks orthogonal to the model's fine-tuning (e.g., domain-specific jargon, personality modulation).
- Settings where prompt tokens are limited (e.g., low-latency agents with small context budgets).

7.2 The Tokenizer Bottleneck

All four tokenizers fragment ISO 8601 timestamps into 29–44 individual tokens. This creates a fundamental challenge for both steering and prompting: the model must use multi-head attention to reconstruct temporal semantics from scattered character-level tokens. This suggests that **temporal-aware tokenization** (treating date/time patterns as single tokens) could significantly improve both approaches.

7.3 Mid-Layer Steering as Future Direction

Our contrastive analysis shows that cosine distance *increases* at earlier layers (0.168 at layer 26 vs. 0.151 at layer 35), suggesting that mid-layer representations may offer more leverage for steering. Targeting layers 15–25, where representations are transitioning from syntactic to semantic, could yield more effective steering vectors. This aligns with findings from representation engineering [9] and the bias-only adaptation work [2], which found optimal steering at intermediate layers.

8 Conclusion

We presented a systematic comparison of activation steering and prompt engineering for agentic calendar tasks. Our tokenizer analysis reveals universal fragmentation of temporal expressions across four model families. Contrastive feature extraction and logit lens analysis confirm that Qwen3-4B develops clear calendar-related features in its final layers, promoting tokens like `schedule`, `agenda`, and `attendees`.

However, our central finding is that **last-layer steering is ineffective on instruction-tuned models** for tasks within their fine-tuning distribution. The instruction tuning has already aligned the model's behavior, making the steering vector redundant. This is a cautionary result for practitioners considering steering as a plug-and-play enhancement for instruct models.

Future work includes: (1) mid-layer steering experiments targeting layers 15–25, (2) full inference evaluation with Ollama/llama.cpp integration, (3) steering on base (non-instruct) model variants, (4) training SAE features specifically for calendar tasks using the Neuronpedia framework, and (5) investigating budget guidance [3] for controlling reasoning length in the agentic loop.

All code, data, and steering vectors are available at `[repository URL]`.

Acknowledgments

Analysis performed using the Transformer Explainer tool [10] and Neuronpedia interpretability platform. Model weights courtesy of the Qwen team [11].

References

[1] A. Turner, L. Thiergart, D. Udell, G. Leech, U. Mini, and M. MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.

[2] L. Gao et al. Steering LLM reasoning through bias-only adaptation. In *EMNLP*, 2025. arXiv:2505.18706.

[3] J. Li, W. Zhao, Y. Zhang, and C. Gan. Steering LLM thinking with budget guidance. *arXiv preprint arXiv:2506.13752*, 2025.

[4] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[5] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

[6] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[7] T. Schick et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

[8] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *NeurIPS*, 2024.

[9] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A. Dombrowski, S. Goel, N. Li, M. Lin, S. Wang, C. Xiao, J. Zou, Z. Kolter, and D. Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

[10] H. Cho et al. Transformer Explainer: Interactive learning of text generative models. 2024. https://poloclub.github.io/transformer-explainer/

[11] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.